

MANUAL PRACTICO — DIA 2

Automatizacion en AWS

CodeBuild · CodePipeline · Inspector

Requisitos previos

Antes de empezar, asegurate de tener:

- Node.js 20 o superior instalado. Verifica con `node --version`.
- Una cuenta de GitHub y Git configurado en tu maquina.
- Una cuenta de AWS activa (los ejercicios 2B y 3 usan la consola de AWS).
- El proyecto `servicio-tienda` de la carpeta `dia2/proyecto/`.

Sin cuenta AWS

Los ejercicios 1 y 4 corren 100% en local y no necesitan cuenta AWS. Si no tienes cuenta, puedes seguir todo el manual leyendo las capturas de los ejercicios 2B y 3, que muestran exactamente lo que veras en la consola.

Ejercicio 1 — El pipeline local

Antes de subir nada a la nube, corremos el pipeline en nuestra maquina. Mismas fases, mismo resultado. Si pasa aqui, pasa en la nube; si falla aqui, lo arreglamos gratis.

1 Preparar el proyecto

- Abre una terminal en `dia2/proyecto/servicio-tienda`.
- Instala dependencias: `npm install` (Faker + json-server).

2 Correr el pipeline

- Ejecuta: `npm run pipeline`.

El runner ejecuta las 5 fases del pipeline, las mismas que definiremos en el `buildspec.yml` de AWS:

Fase	Que hace
INSTALL	npm install: instala las dependencias
PRE_BUILD	genera los datos con Faker, restaura el seed y levanta la API con un healthcheck
BUILD	corre las 3 suites de tests (186 aserciones en total)
POST_BUILD	muestra el reporte de resultados y detiene la API
SECURITY_GATE	ejecuta el analisis de AWS Inspector (ver Ejercicio 4)

El resultado final es **PIPELINE PASSED** si los 186 tests pasan.

```

— FASE: INSTALL —————
OK  Dependencias instaladas
— FASE: PRE_BUILD —————
OK  Datos generados (2-servicio/db.seed.json)
OK  API responde en http://localhost:3001
— FASE: BUILD (corriendo tests) —————
OK  Total pasaron : 186   fallaron: 0
— FASE: POST_BUILD —————
OK  Reporte en 4-pipeline/test-report.json
— FASE: SECURITY_GATE (AWS Inspector) —————
->  Gate informativo (no bloquea por defecto)
=====
OK  PIPELINE PASSED

```

3 Reto: romper el build

- Abre `2-servicio/db.json` y cambia el precio del producto 1 a `-99`. Guarda.
- Corre `npm run pipeline`. El build ahora falla: el precio no puede ser negativo.
- Restaura el estado limpio con `npm run seed`.

Ejercicio 2A — Subir el proyecto a GitHub

AWS CodeBuild y CodePipeline leen el código desde GitHub. Primero subimos el proyecto.

- Crea un repositorio nuevo en GitHub llamado `servicio-tienda` (puede ser privado).
- Desde la carpeta del proyecto, inicializa y sube:

```
git init
git add .
git commit -m "Servicio tienda: pipeline QA"
git branch -M main
git remote add origin https://github.com/<usuario>/servicio-tienda.git
git push -u origin main
```

- Recarga tu repo en GitHub. Confirma que ves las carpetas `1-generar-datos ... 5-inspector` y el `package.json`, pero NO ves `node_modules` ni archivos de datos (los ignora el `.gitignore`).

Que NO se sube

El `.gitignore` evita subir `node_modules/`, los datos generados por Faker (`db.seed.json`, `db.json`) y los reportes (`test-report.json`, `resumen-analisis.json`). Todo eso se regenera en el pipeline.

5. Ejercicio 2B — Crear el proyecto en AWS CodeBuild

CodeBuild es "el cocinero": sabe preparar el plato ejecutando el buildspec.yml en un contenedor en la nube. Aquí creamos el proyecto que corre nuestros 186 tests.

1 (Opcional) Crear un bucket S3 para los reportes

Si quieres guardar el reporte de tests como artefacto, crea primero un bucket S3. Es opcional: sin esto, los logs igual aparecen en CloudWatch.

Crear bucket Información

Los buckets son contenedores de datos almacenados en S3.

Configuración general Información

Región de AWS: SE-USA, Gila Norte de Virginia us-east-1

Tipo de bucket: **Información**

- ☒ **Uso general**
Recomendado para la mayoría de los casos de uso y patrones de acceso. Los buckets de uso general son el tipo de bucket de S3 original. Tienen una combinación de lista de acceso controlado que permite algunos tipos de permisos estándar por defecto (disponibilidad).
- ☐ **Desahilitado**
Recomendado para casos de uso de alta seguridad. Este bucket utiliza únicamente la clase de almacenamiento S3 Intelligent Tiering, que proporciona un almacenamiento más rápido de los datos dentro de una única zona de disponibilidad.

Espacio de nombres del bucket Información

El espacio de nombres es el que define cómo se llaman los buckets. [Más información](#)

- ☒ **Espacio de nombres global**
No tiene precedentes. El uso de buckets de uso general en el espacio de nombres global.
- ☐ **Espacio de nombres regional de la cuenta (experimental)**
Los buckets de uso general creados en el espacio de nombres regional de la cuenta son exclusivos de esa cuenta. Este bucket no se puede crear con otros puntos de AWS.

Nombre del bucket: Información

tiendatestreports

El nombre de un bucket debe tener entre 3 y 63 caracteres y no puede tener más de 255 caracteres de longitud global. Los nombres de los buckets también deben comenzar y terminar con una letra o número. Los caracteres válidos son a-z, 0-9, guiones (-) y guiones bajos (_). [Más información](#)

Exportar la configuración del bucket existente (opcional)

Sube un archivo de configuración del bucket en los siguientes formatos:

[Omitir el bucket](#)

[Ver bucket](#) [Ver bucket](#)

Propiedad de objetos Información

Controla la propiedad de los objetos dentro de este bucket desde las cuentas de AWS y el uso de listas de control de acceso (ACL). La propiedad de los objetos determina quién puede modificar el acceso a los objetos.

Propiedad del objeto

- ☒ **ACL desahabilitadas (recomendado)**
Todos los objetos de este bucket son propiedad de esta cuenta. El acceso a este bucket y sus objetos se controla solo mediante permisos.
- ☐ **ACL habilitadas**
Los objetos de este bucket pueden ser propiedad de otras cuentas de AWS. El acceso a este bucket y sus objetos se controla mediante ACL.

Propiedad del objeto

Aplicado al propietario del bucket:

S3 > Crear bucket: nombre unico global (ej. tiendatestreports), tipo Uso general, ACL desahilitadas (recomendado).

Configuración de bloqueo de acceso público para este bucket

Se otorga acceso público a los buckets y objetos a través de listas de control de acceso (ACL), políticas de bucket, políticas de acceso o todas las anteriores. A fin de garantizar que se bloquee el acceso público a todos los buckets y objetos, active **Bloquear todo el acceso público**. Esta configuración se aplica exclusivamente a este bucket y a sus partes de acceso. AWS recomienda activar **Bloquear todo el acceso público**, pero, antes de aplicar cualquiera de estos ajustes, asegúrese de que las aplicaciones funcionando correctamente no usen acceso público. Si necesita cierto nivel de acceso público a los buckets o objetos, puede personalizar la configuración individual a continuación para adaptarla a sus casos de uso de almacenamiento específicos. [Más información](#)

☒ **Bloquear todo el acceso público**

Active esta configuración obligatoria para evitar los cuatro escenarios que permiten el acceso público. Cada uno de los siguientes ajustes son independientes de los otros:

- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de reglas listas de control de acceso (ACL)**
El bloqueo de la propiedad de acceso público se aplica a todos los buckets, agregando automáticamente, a nivel de la cuenta de acceso, ACL de acceso público para buckets y objetos existentes. Si la configuración de bloqueo de acceso público se activa, los permisos de acceso público se eliminan automáticamente de todos los ACL.
- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de cualquier lista de control de acceso (ACL)**
El bloqueo de acceso público se aplica a todos los buckets y objetos existentes. Si la configuración de bloqueo de acceso público se activa, los permisos de acceso público se eliminan automáticamente de todos los ACL.
- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de políticas de bucket y partes de acceso públicas numeradas**
El bloqueo de acceso público se aplica a todos los buckets y objetos existentes. Si la configuración de bloqueo de acceso público se activa, los permisos de acceso público se eliminan automáticamente de todas las políticas de bucket y partes de acceso públicas numeradas.
- ☐ **Bloquear el acceso público a partes numeradas a buckets y objetos concedido a través de cualquier política de bucket y partes de acceso pública**
El bloqueo de acceso público se aplica a todos los buckets y objetos existentes. Si la configuración de bloqueo de acceso público se activa, los permisos de acceso público se eliminan automáticamente de todas las políticas de bucket y partes de acceso pública.

Control de versiones de buckets

Si quiere de versiones en una forma de mantener múltiples versiones de un objeto dentro del mismo bucket. Puede utilizar el control de versiones para conservar, recuperar y restaurar todas las versiones de los objetos almacenados en su bucket de Amazon S3. Con el control de versiones, puede recuperar una versión de los objetos eliminados de los buckets y de los objetos en las aplicaciones. [Más información](#)

Control de versiones de buckets

- ☒ **Desactivar**
- ☐ **Habilitar**

Etiquetas - opcional

Puede usar etiquetas de bucket para analizar los permisos de un bucket, además de administrarlos y exportarlos. [Más información](#)

- ☒ **Puede usar las API de ListTagsForResource, GetTagResource y UntagResource para administrar las etiquetas en los buckets de uso general de S3 para el control de acceso, además de la asignación de contenedores y la organización de los recursos.** Para garantizar una transición fluida, asegúrese primero de realizar las acciones de ListTagsForResource, GetTagResource y UntagResource. [Más información](#)

No hay etiquetas asociadas a este bucket.

[Agregar nueva etiqueta](#)

Puede agregar hasta 50 etiquetas.

Deja "Bloquear todo el acceso público" ACTIVADO: el reporte es interno. Control de versiones en Desactivar.

Cifrado predeterminado Información

El cifrado del lado del servidor se aplica automáticamente a los nuevos objetos almacenados en este bucket.

Tipo de cifrado Información

Elige un tipo de cifrado de predeterminado. Para obtener más información sobre los precios, consulta [AWS KMS pricing](#) (cifrado de SSE-KMS en la pestaña Storage (Recomendamos el [precio de precios de recursos](#))).

- ☒ Cifrado del servidor con claves administradas de Amazon S3 (SSE-S3)
- ☐ Cifrado del servidor con claves de AWS Key Management Service (SSE-KMS)
- ☐ Cifrado de doble capa del servidor con claves de AWS Key Management Service (SSE-KMS)

Clave de bucket

El uso de una clave de bucket de SSE-KMS reduce los costos de cifrado y reduce las llamadas a AWS KMS. Las claves de bucket de SSE no son compatibles con SSE-KMS. [Ver detalles](#)

- ☐ Desactivar
- ☒ Habilitar

Configuración avanzada

Después de crear el bucket, puede cargar archivos y carpetas, y configurar ajustes adicionales en él.

Crear bucket

Cifrado SSE-S3 (por defecto). Baja y pulsa "Crear bucket".

2 Crear el proyecto de CodeBuild

Ve a la consola de AWS > [CodeBuild](#) > **Create build project**.

Paso 1 — Configuración del proyecto y origen

- Nombre del proyecto: `servicio-tienda-tests`.
- Origen (Source): proveedor **GitHub**. La primera vez, pulsa "Conectar a GitHub" y autoriza la conexión (GitHub App administrada por AWS). Selecciona el repositorio `<usuario>/servicio-tienda`.

Editar proyecto

Configuración del proyecto

Nombre del proyecto

`servicio-tienda-tests`

Tipo de proyecto

Selecciona el tipo de proyecto que deseas crear. [Ver detalles](#)

- ☒ Proyecto predeterminado
Crea un proyecto de CodeBuild predeterminado.
- ☐ Proyecto de ejecución
Crea un proyecto personalizado de CodeBuild para usar reglas de trabajo en Actions de GitHub, acciones de GitHub Enterprise, GitHub Actions.

Configuración adicional

Descripción, acceso público a la compilación, imagen de compilación, límites de compilación simultáneos, Etiquetas

Origen

Origen 1: principal

Proveedor del origen

GitHub

Credencial

☒ Su cuenta se conecta correctamente mediante una aplicación GitHub administrada por AWS. Adminístrala las credenciales de la cuenta.

☐ Usa las credenciales de ejecución únicamente para este proyecto.

Repositorio

☒ Repositorio en mi cuenta de GitHub

☐ Repositorio público

☐ Webhook con ámbito de GitHub

Repositorio

`https://github.com/<usuario>/servicio-tienda.git`

Versión de origen (opcional) Información

Proporciona una versión de control de código, una referencia de código, un ID de confirmación, una etiqueta o una referencia y un ID de confirmación.

Configuración adicional

Propiedades del estado de Git, Submódulos de Git, Configuración de estado de compilación

Nombre del proyecto + Origen GitHub. La credencial muestra "Su cuenta se conecta correctamente". Repositorio: en mi cuenta de GitHub.

Paso 2 — Entorno (el contenedor donde corre)

- Imagen administrada, sistema operativo **Amazon Linux**, runtime Standard, imagen `aws/codebuild/amazonlinux-x86_64-standard:6.0` (trae Node 20). Rol de servicio: New service role.

Eventos de webhook de origen principal [Información](#)

Webhook - opcional [Información](#) ⓘ

☐ Volver a compilar cada vez que un cambio de código se envía a este repositorio

▼ **Entorno**

Método de aprovisionamiento [Información](#) ⓘ

☒ **Rápido elementa**
Aprovisiona automáticamente la infraestructura de compilación en respuesta a los nuevos compromisos.

☐ **Capacidad reservada**
Utiliza una Flota de Instancias de Spot para las compilaciones. El tipo de entorno e información de flotas se actualiza para el proyecto.

Imagen del entorno

☒ **Imagen administrada**
Utiliza una imagen administrada por AWS. [Ver todas las imágenes](#)

☐ **Imagen personalizada**
Proporciona una imagen de Docker.

Computación

☒ **EC2**
Optimizado para obtener flexibilidad durante las ejecuciones de acciones.

☐ **Lambda**
Optimizado para mayor elasticidad y menores el tiempo de inicio de las acciones de flujo de trabajo.

Modo de ejecución

☒ **Contenedor**
Se ejecuta en los contenedores de Docker.

☐ **Instancia**
Ejecución directa en una instancia de EC2.

Sistema operativo

Amazon Linux

Tiempo de ejecución

Standard

Imagen

aws/codebuild/amazonlinux-x86_64-standard:6.0

Versión de la imagen

Utilizar siempre la imagen más reciente para esta versión del tiempo de ejecución.

☐ Utilizar computación acelerada por GPU

Imagen administrada · EC2 · Amazon Linux · Standard · imagen amazonlinux-x86_64-standard:6.0 (Node 20).

Paso 3 — Buildspec y artefactos

- Especificación de compilación: "Utilizar un archivo de buildspec". Nombre: `4-pipeline/buildspec.yml` (ruta EXACTA dentro del repo).
- Artefactos (opcional): tipo Amazon S3, elige el bucket que creaste.

► **Configuración adicional**
Tiempo de espera, precalentado, certificado, VPC, tipo de computación, variables de entorno, nombres de archivos, notificaciones automáticas, contenido de registro.

▼ **Especificación de compilación**

Especificaciones de la compilación

☐ **Insertar comandos de compilación**
Inyectar los comandos de compilación como la configuración del proyecto de compilación.

☒ **Utilizar un archivo de especificación de compilación**
Inyectar los comandos de compilación en un archivo de especificación de compilación con formato YAML.

Nombre de la especificación de compilación - opcional
De forma predeterminada, CodeBuild usa el nombre de archivo de especificación de compilación `buildspec.yml` en el directorio raíz del código fuente. Si su archivo de especificación de compilación utiliza un nombre o una ubicación diferentes, incluya aquí desde la raíz de origen aquí (por ejemplo, `buildspec-test.yml` o `configuration/buildspec.yml`).

4-pipeline/buildspec.yml

▼ **Configuración del lote**

☐ **Definir la configuración del lote - opcional**
También puede definir o actualizar la configuración del lote al iniciar un lote de compilación.

▼ **Artefactos** Guardar artefacto

Artefacto 1: principal

Tipo

Amazon S3

Puede optar por no utilizar artefactos o guardarlos en una imagen de Docker en Amazon ECR.

Nombre del bucket

serviciotiendareporte

Nombre
El nombre de la carpeta o del archivo comprimido en el bucket que contendrá los artefactos de salida. Utilice "Empaquetado de artefactos" en la sección "Configuración adicional" para elegir si desea utilizar como carpeta o un archivo comprimido. Si no se proporciona un nombre, se asigna de forma predeterminada al nombre del proyecto.

SERVICIO-TIENDA-TEST-2

Buildspec: "Utilizar un archivo" con nombre 4-pipeline/buildspec.yml. Artefacto: Amazon S3.

Paso 4 — Registros (para VER el reporte)

- Baja hasta **Registros** y marca **CloudWatch Logs**. Grupo: `aws/codebuild/servicio-tienda-tests`.

☐ Habilitar el control de versiones automático
Utilizar el nombre del artefacto especificado en el archivo de configuración de compilación.

Nota - opcional
Escriba el nombre S3 de la carpeta de salida de la compilación.
Ejemplo: MyPath/MyArtifact.s3

Tipo de espacio de nombres - opcional
Ninguno
Ejemplo: "El nombre de la compilación" para mostrar al ID de compilación en la ruta al artefacto S3 o la carpeta de salida de la compilación, ej., MyPath/MyArtifactID/MyArtifact.s3. Si lo desea, seleccione "Ninguno".

Enmascaramiento de artefactos
☒ Ninguno
 Los artefactos no se enmascaran al cargarlos en el bucket.
☐ S3
 AWS CodeBuild cargará artefactos de un artefacto comprendido que se enmascara al bucket especificado.

☐ Deshabilitar el cifrado de artefactos
Desactivar el cifrado de los artefactos para sustituir un cifrado estándar o para conservar los datos con otro proveedor.

Configuración adicional
Cache, clave de cifrado

▼ Registros
CloudWatch

☒ CloudWatch Logs - opcional
Al marcar esta opción, se creará un registro de compilación de salida en CloudWatch.

Nombre del grupo - opcional
aws-logs-9-9-us-east-1-123456789012-us-east-1
El nombre del grupo de los registros de CloudWatch Logs. El nombre del grupo de registros será aws-logs-9-9-us-east-1-123456789012-us-east-1.

Prefijo del nombre del flujo - opcional
ss
El prefijo del nombre del flujo en CloudWatch Logs.

ss

☒ Registros de S3 - opcional
Marcar esta opción cargará los registros de compilación de salida en S3.

Registros > CloudWatch Logs marcado. ESTE es el paso que suele faltar: sin el, la pestaña de registros sale vacía.

☒ Registros de S3 - opcional
Marcar esta opción cargará los registros de compilación de salida en S3.

Bucket
elasticbeanstalk-us-east-1-123456789012-us-east-1

Prefijo de ruta
report

☐ Deshabilitar el cifrado de registros de S3

▼ Permisos de rol de servicio
Rol de servicio
Ejemplo: arn:aws:iam::123456789012:role/service-role/elasticbeanstalk-ec2-role

☒ Permitir que AWS CodeBuild modifique este rol de servicio a fin de que se pueda utilizar con este proyecto de compilación.

Cancelar Actualizar proyecto

Opcional: Registros de S3 (bucket + prefijo). Rol de servicio: New service role. Pulsa Crear/Actualizar proyecto.

Que se espera ver al correr el build

En la página del proyecto pulsa **Start build**. En "Build logs" veras las fases INSTALL → PRE_BUILD → BUILD → POST_BUILD. En BUILD: 186 pasaron 0 fallaron. Al terminar: estado **Succeeded** en verde.

6. Ejercicio 3 — Crear el pipeline en AWS CodePipeline

Si CodeBuild es el cocinero, CodePipeline es el restaurante completo: sabe cuando llamar al cocinero, que hacer si el plato sale mal y como llevarlo a la mesa (deploy).

Requisito previo

Este ejercicio REUTILIZA el proyecto de CodeBuild [servicio-tienda-tests](#) del Ejercicio 2B. CodePipeline no corre tests por si mismo: en su etapa Build LLAMA a ese proyecto de CodeBuild. Si no lo creaste, vuelve al Ejercicio 2B primero.

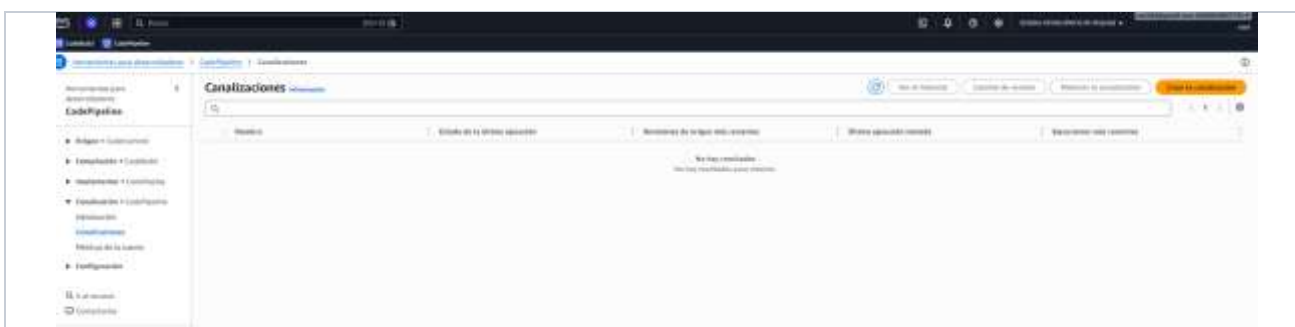
La conexion a GitHub (GitHub App)

Tanto CodeBuild como CodePipeline hablan con GitHub mediante una **conexion de tipo GitHub App** (AWS Developer Tools > Connections). La primera vez debes crearla: Crear conexion > GitHub > autorizar, y darle acceso al repositorio. Si un push no dispara el pipeline, revisa que la conexion este en estado **Available** (no Pending). Con GitHub App el disparador NO aparece en GitHub > Settings > Webhooks: es normal, usa su propio canal.

6.1 Crear la canalizacion paso a paso

Paso 1 — Abrir CodePipeline

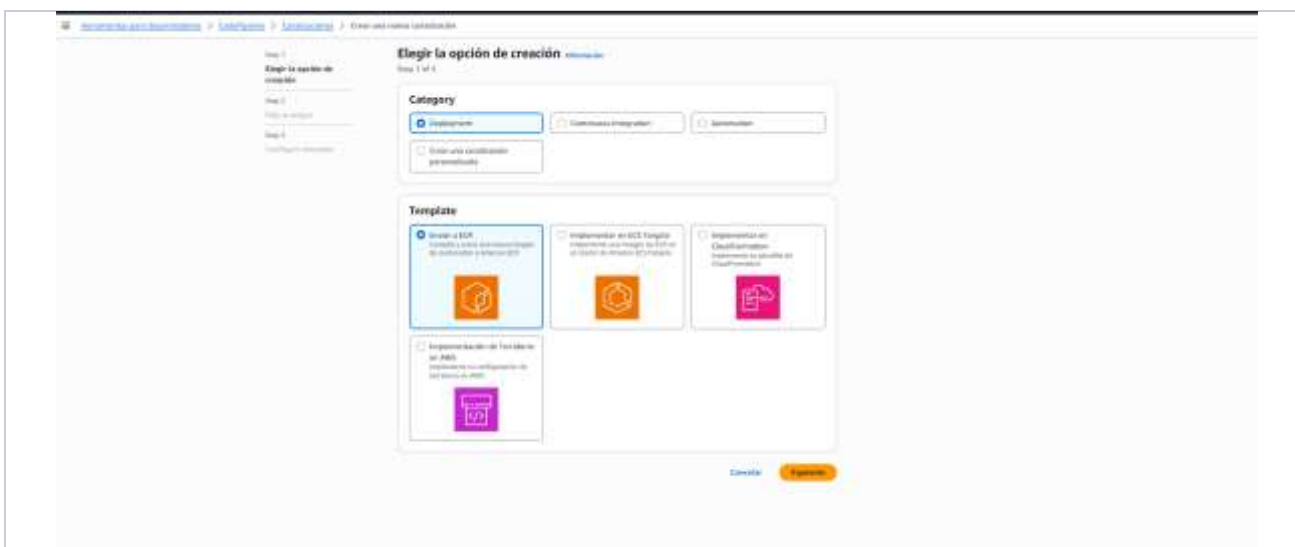
- Consola AWS > CodePipeline > Canalizaciones > **Crear la canalizacion**.



CodePipeline > Canalizaciones. Pulsa el boton naranja "Crear la canalizacion".

Paso 2 — Elegir la opcion de creacion (la trampa esta aqui)

- AWS muestra plantillas (ECR, ECS Fargate, CloudFormation, Terraform). **NO elijas ninguna plantilla.**



AWS ofrece plantillas de despliegue. No aplican a este ejercicio.

- En Category selecciona **"Crear una canalizacion personalizada"**. Recien ahi el asistente pasa a los pasos de origen, compilacion, prueba e implementacion.

El screenshot muestra la interfaz de usuario de AWS CodePipeline en el paso 'Elegir la opción de creación' (Step 1 of 7). A la izquierda hay un menú de navegación con los pasos: Step 1 (Elegir la opción de creación), Step 2 (Elegir la configuración de la canalización), Step 3 (Agregar la etapa de origen), Step 4 - opcional (Agregar la etapa de compilación), Step 5 - opcional (Agregar la etapa de prueba), Step 6 (Agregar la etapa de implementación) y Step 7 (Revisar). El contenido principal muestra el título 'Elegir la opción de creación' con un enlace a 'Información'. Debajo, en la sección 'Category', hay tres opciones de radio: 'Deployment', 'Continuous Integration' y 'Automated'. La opción 'Crear una canalización personalizada' está seleccionada y resaltada con un recuadro azul. En la parte inferior derecha, hay dos botones: 'Cancelar' (azul) y 'Siguiente' (naranja).

Selecciona "Crear una canalizacion personalizada".

Paso 3 — Configuración de la canalización

- Nombre: **tienda-pipeline**. Modo de ejecución: En cola. Rol de servicio: Nuevo rol (deja marcado "Permitir que AWS CodePipeline cree un rol").

El screenshot muestra el paso 'Elegir la configuración de la canalización' (Step 2 of 7) en AWS CodePipeline. El menú de la izquierda es similar al anterior. El título principal es 'Elegir la configuración de la canalización' con un enlace a 'Información'. La sección 'Configuración de la canalización' contiene:

- Nombre de la canalización:** Un campo de texto con el valor 'tienda-pipeline' y una advertencia de límite de caracteres.
- Modo de ejecución:** Tres opciones de radio: 'Reemplazadas', 'En cola' (seleccionada) y 'En paralelo'.
- Rol de servicio:** Dos opciones de radio: 'Nuevo rol de servicio' (seleccionada) y 'Rol de servicio existente'.
- Nombre del rol:** Un campo de texto con el valor 'AWSCodePipelineServiceRole-us-east-1-tienda-pipeline'.
- Un checkbox 'Permitir que AWS CodePipeline cree un rol de servicio para que pueda utilizarse con esta nueva canalización' que está marcado.

 En la parte inferior, hay tres botones: 'Cancelar' (azul), 'Anterior' (azul) y 'Siguiente' (naranja).

Nombre tienda-pipeline · modo En cola · Nuevo rol de servicio.

Paso 4 — Etapa de origen (Source)

- Proveedor: **GitHub (a través de GitHub App)**. Conexión: la que autorizaste. **Nombre del repositorio:** `<usuario>/servicio-tienda` (formato usuario/repo, NO la URL). Rama: `main`. Webhook push/pull activado.

Step 1
Elegir la opción de creación

Step 2
Elegir la configuración de la canalización

Step 3
Agregar la etapa de origen

Step 4 - opcional
Agregar la etapa de compilación

Step 5 - opcional
Agregar la etapa de prueba

Step 6
Agregar la etapa de implementación

Step 7
Revisar

Agregar la etapa de origen Información

Step 3 of 7

Origen

Proveedor del origen
Aquí es donde almacenó sus artefactos de entrada para la canalización. Elija el proveedor y luego proporcione los detalles de conexión.

GitHub (a través de GitHub App)

Conexión
Elija una conexión existente que ya haya configurado o cree una nueva; luego, vuelva a esta tarea.

aws:awscodeconnections:us ☒ ☐ Conectarse a GitHub

Nombre del repositorio
Elija un repositorio en la cuenta de GitHub.

<usuario>/servicio-tienda

Puede escribir o pegar la ruta del grupo en cualquier proyecto al que puedan acceder las credenciales proporcionadas. Use el formato <grupo/subgrupo/proyecto>.

Ramificación predeterminada
La ramificación predeterminada solo se usará cuando la ejecución de la canalización comience desde una fuente diferente o se inicie manualmente.

main

Formato de artefacto de salida
Elija el formato del artefacto de salida.

☒ **CodePipeline predeterminado**
AWS CodePipeline utiliza el formato zip predeterminado para los artefactos de la canalización. No incluye metadatos de Git sobre el repositorio.

☐ **Con completo**
AWS CodePipeline transmite metadatos sobre el repositorio que permiten que las acciones posteriores generen un clon de Git completo. Solo se admite para acciones de AWS CodeBuild. [Más información](#)

☒ Habilitar el reintento automático en caso de fallo de una etapa

Eventos de webhook

Webhook - opcional

☒ Inicie su canalización en eventos de solicitud push y pull.

► Filtros de eventos de webhook - opcional

[Eliminar filtros](#)

Origen GitHub (GitHub App). Nombre del repositorio en formato `<usuario>/servicio-tienda`. Rama `main`.

Error mas comun

En "Nombre del repositorio" escribe `usuario/repo`, NO la URL completa. Si pones `https://github.com/<usuario>/servicio-tienda.git` el stage Source falla con: `"No Branch [main] found for FullRepositoryName"`. AWS no identifica el repo por la URL, solo por el formato usuario/repo.

Paso 5 — Etapa de compilacion (Build): llama a CodeBuild

- Proveedor: **Otros proveedores de compilacion > AWS CodeBuild**. Nombre del proyecto: `servicio-tienda-tests` (el del Ejercicio 2B). Tipo: Compilacion unica. Artefacto de entrada: SourceArtifact.

Elegir la configuración de la canalización

Step 3: Agregar la etapa de origen

Step 4 - opcional: Agregar la etapa de compilación

Step 5 - opcional: Agregar la etapa de prueba

Step 6: Agregar la etapa de implementación

Step 7: Revisar

Proveedor de compilación
Elija la herramienta que desea usar para ejecutar los comandos de compilación y especifique los artefactos para la acción de compilación.

☐ Comandos ☒ Otros proveedores de compilación

AWS CodeBuild

Nombre del proyecto
Elija un proyecto de compilación que ya haya creado en la consola de AWS CodeBuild. O bien, cree un proyecto de compilación en la consola de AWS CodeBuild y luego vuelva a esta tarea.

SERVICIO-TIENDA-TEST-2

☐ Definir la anulación de la especificación de compilación - opcional
Archivo o definición de especificación de compilación que anula el último definido en el proyecto de compilación, solo para esta compilación.

Anulación del rol de servicio - opcional
Elija un rol de IAM que anule el rol de servicio predeterminado del proyecto de CodeBuild para esta acción.

Variables del entorno - opcional
Elija la clave, el valor y el tipo para las variables de entorno de CodeBuild. En el campo de valor, puede hacer referencia a las variables generadas por CodePipeline. [Más información](#)

Tipo de compilación

☒ Compilación única
Activa una compilación única.

☐ Compilación por lotes
Activa varias compilaciones como una sola ejecución.

Región
Estados Unidos (Norte de Virginia)

Artefactos de entrada
Elija un artefacto de entrada para esta acción. [Más información](#)

SourceArtifact
Definido por: Source

☒ Habilitar el reintento automático en caso de fallo de una etapa

Compilacion: AWS CodeBuild, proyecto servicio-tienda-tests, artefacto de entrada SourceArtifact.

Aqui esta la conexion entre los dos ejercicios

El stage Build NO tiene logica propia de tests: llama al proyecto CodeBuild del 2B, que ya sabe correr el buildspec con los 186 tests. CodePipeline solo decide CUANDO llamarlo.

Paso 6 — Etapas de prueba e implementacion: OMITIR ambas

- Etapa de prueba (opcional): **Omitir la etapa de prueba** (los tests ya corren dentro del Build).

Step 1: Elegir la opción de creación

Step 2: Elegir la configuración de la canalización

Step 3: Agregar la etapa de origen

Step 4 - opcional: Agregar la etapa de compilación

Step 5 - opcional: **Agregar la etapa de prueba**

Step 6: Agregar la etapa de implementación

Step 7: Revisar

Agregar la etapa de prueba - opcional Información

Step 5 of 7

Prueba - opcional

Proveedor de pruebas
Elija cómo desea probar la aplicación o el contenido. Elija el proveedor y, a continuación, proporcione los detalles de configuración de dicho proveedor.

☒ Habilitar el reintento automático en caso de fallo de una etapa

Cancelar Anterior Omitir la etapa de prueba Siguiente

Step 5 Prueba (opcional): pulsa "Omitir la etapa de prueba".

- Etapa de implementación (opcional): **Omitir la etapa de implementación** (no desplegamos en esta demo).

Step 1: Elegir la opción de creación

Step 2: Elegir la configuración de la canalización

Step 3: Agregar la etapa de origen

Step 4 - opcional: Agregar la etapa de compilación

Step 5 - opcional: Agregar la etapa de prueba

Step 6: **Agregar la etapa de implementación**

Step 7: Revisar

Agregar la etapa de implementación Información

Step 6 of 7

Implementar - opcional

Proveedor de implementación
Elija cómo desea implementar la aplicación o el contenido. Elija el proveedor y, a continuación, proporcione los detalles de configuración de ese proveedor.

☒ Configurar la reversión automática en caso de error en una etapa

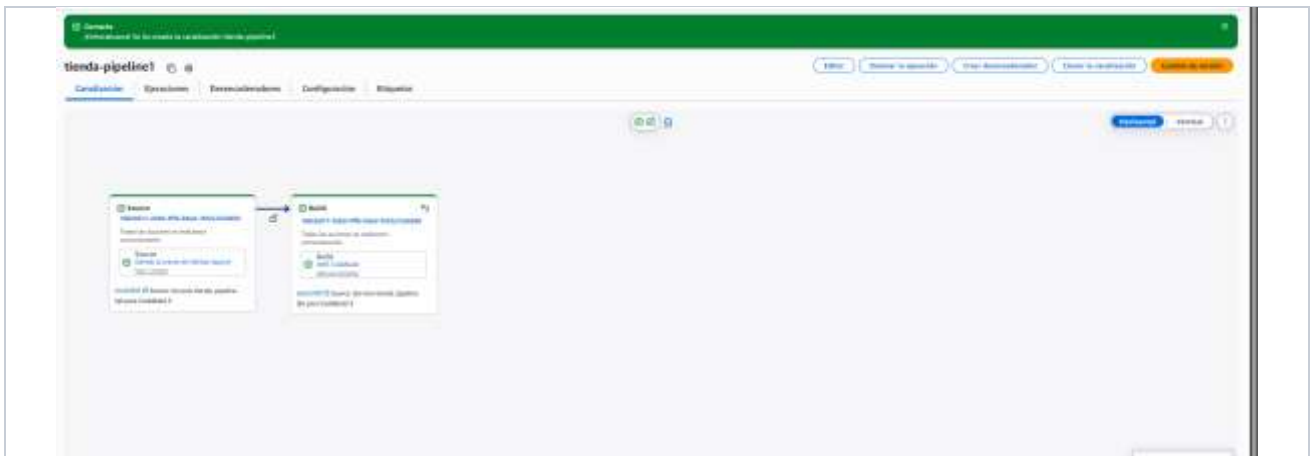
☐ Habilitar el reintento automático en caso de fallo de una etapa

Cancelar Anterior Omitir la etapa de implementación Siguiente

Step 6 Implementacion (opcional): pulsa "Omitir la etapa de implementacion".

Paso 7 — Revisar y crear

- Revisar > **Crear la canalización**. Apenas se crea, el pipeline arranca solo.



Pipeline creado: Source y Build en verde. "Todas las acciones se realizaron correctamente".

6.2 Como validar y probar el pipeline

- Tras crearlo, el pipeline ejecuta Source → Build automáticamente. Espera a que ambos queden en verde.
- Prueba el disparador: haz un cambio pequeño en el repo, `git commit` y `git push`. En segundos el pipeline debe arrancar una nueva ejecución por sí solo.
- Entra al stage Build > Ver detalles/registros: veras el mismo log de CodeBuild con las fases y los 186 tests.

7. Ejercicio 4 — El security gate con AWS Inspector

AWS Inspector v2 escanea vulnerabilidades en dependencias, imagenes de contenedor y sistema operativo. En este ejercicio analizamos un reporte y lo convertimos en una **compuerta de seguridad** dentro del mismo pipeline.

Este caso NO es aislado

Es la 5a fase del MISMO pipeline del Ejercicio 2B/3. Igual que el BUILD corre los 186 tests, el SECURITY_GATE corre el analisis de Inspector. Los tests dicen si la logica funciona; el gate dice si las dependencias son seguras. Misma tuberia, dos compuertas.

7.1 Analizar el reporte

- Desde `servicio-tienda`, corre: `npm run inspector`. Veras el dashboard de severidades, la lista priorizada y el plan de accion top 5.
- Lee el score de riesgo: formula `critico*10 + alto*5 + medio*2 + bajo*1`. Con los 12 hallazgos del reporte el score es **50** > umbral 25 → **DEPLOY BLOQUEADO**.

```
CRITICO  ####...  2  (17%)
ALTO    ####...  4  (33%)
MEDIO   ####...  4  (33%)
BAJO    ##.....  2  (17%)
```

- 1) [CRITICO 10] Log4Shell - RCE en Log4j (CVE-2021-44228)
- 2) [CRITICO 9.8] Spring4Shell - RCE (CVE-2022-22965)

Score: 50 (umbral bloqueo=25) -> DEPLOY BLOQUEADO

7.2 El gate dentro del pipeline

- Corre el pipeline completo: `npm run pipeline`. Al final del BUILD veras la fase SECURITY_GATE. Por defecto es **informativa**: avisa del score pero deja pasar (PIPELINE PASSED).
- Activalo como **bloqueante** y vuelve a correr (PowerShell):

```
$env:SECURITY_GATE="on"; npm run pipeline
```

- Ahora los 186 tests pasan, pero el pipeline termina en **SECURITY GATE FAILED** (exit 1): el mismo mecanismo que un test roto, pero por seguridad.

7.3 Reto: ajustar el umbral

- En `5-inspector/analizar-reporte.js` cambia `UMBRAL_BLOQUEO` de 25 a 60. Corre de nuevo con el gate en on: el deploy ya no se bloquea (score 50 < 60).

Debate para el equipo

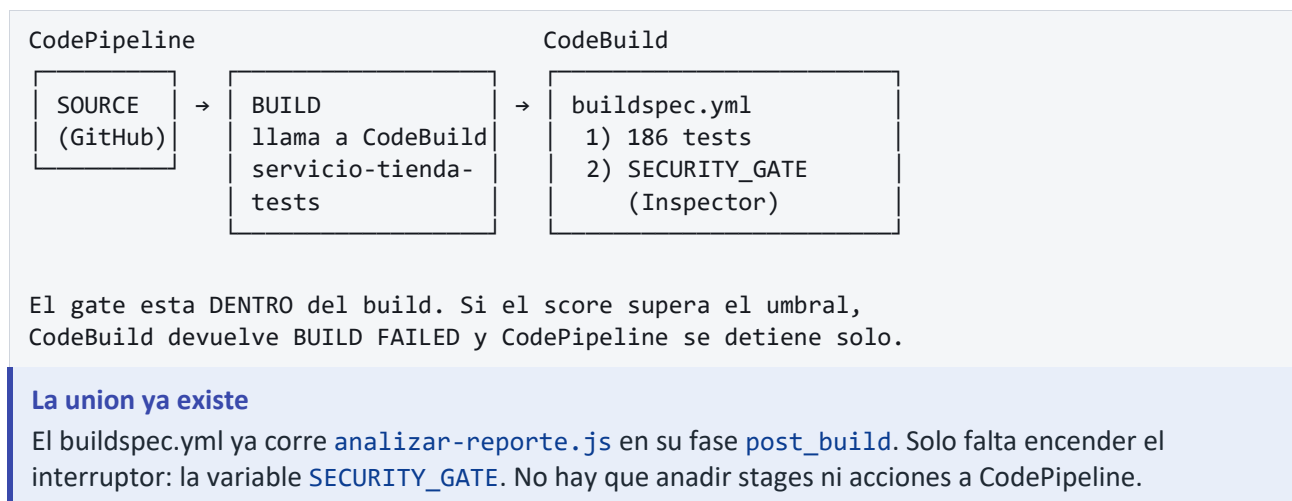
El umbral es una decision de negocio, no tecnica. Hay equipos que bloquean ante cualquier CRITICO, otros que usan un score, otros que solo bloquean si el CVE tiene exploit publico. No hay una respuesta unica.

8. Ejercicio 4B — Unir Inspector al CodePipeline del Ejercicio 3

La pregunta que siempre surge: *¿como conecto Inspector al CodePipeline que ya cree?* La respuesta sorprende: **NO se toca CodePipeline**.

AWS Inspector v2 no es una accion que se arrastra al pipeline: escanea recursos (ECR, EC2, Lambda) de forma continua y asincrona. La forma de convertirlo en compuerta es leer sus hallazgos DENTRO de un build y devolver exit 1. Eso es justo lo que hace nuestro [analizar-reporte.js](#), que ya vive en el buildspec del proyecto de CodeBuild que el pipeline invoca.

8.1 Donde vive el gate (por que no se toca CodePipeline)



8.2 Activar el gate en el proyecto de CodeBuild

- Consola AWS → [CodeBuild](#) → proyecto [servicio-tienda-tests](#) → **Editar** → **Entorno**.
- Despliega **Configuracion adicional** → **Variables de entorno**. Agrega: Nombre `SECURITY_GATE`, Valor `on`. Guarda (Actualizar proyecto).
- Dispara el pipeline: haz un `git push`, o en CodePipeline pulsa **Publicar cambio (Release change)**.
- Mira los logs del stage Build: tras los 186 tests corre el analisis de Inspector. Como el score es $50 > 25$, CodeBuild termina en **BUILD FAILED** y el stage Build de CodePipeline se pone en rojo. El pipeline se detiene: no avanza.
- Para volver a verde: cambia la variable a `off` (gate informativo) o sube el `UMBRAL_BLOQUEO` en [analizar-reporte.js](#), haz push y vuelve a correr.

Es lo mismo que en local

Esto es identico a `$env:SECURITY_GATE="on"; npm run pipeline`. La unica diferencia es DONDE se define la variable: en local es tu terminal; en AWS es la configuracion del proyecto de CodeBuild. El mecanismo (`exit 1` → pipeline detenido) es el mismo.

8.3 La otra ruta: Inspector v2 nativo sobre ECR (nivel produccion)

La integracion anterior mete el analisis dentro del build: rapida, reproducible, ideal para aprender. La ruta "productiva" de AWS es mas pesada. Inspector v2 escanea imagenes de contenedor en ECR de forma continua. El flujo seria:

- Contenerizar la app (un [Dockerfile](#)) y que el pipeline construya la imagen.
- Un stage extra empuja la imagen a un repositorio **ECR** con "scan on push" activado.

- Inspector v2 escanea la imagen automáticamente y publica los hallazgos.
- Un **Lambda** (o un comando en el buildspec via AWS CLI) lee los hallazgos con `inspector2 list-findings`, calcula el score y devuelve exit 0/1 — el mismo gate, pero con datos reales de Inspector.

Por que no lo hacemos en clase

Esta ruta exige contenerizar la app y desplegar a ECR, lo que alarga bastante el ejercicio. El patron, sin embargo, es identico al que ya construiste: leer hallazgos → calcular score → decidir si el pipeline continua.

9. Resumen y comparativa

Lo que construiste en el Día 2:

- Un **servicio bajo prueba** con datos generados por Faker (reproducibles).
- Tres **suites de tests** que validan contrato, reglas de negocio y SLA (186 aserciones).
- Un **pipeline CI/CD** con las fases de CodeBuild, ejecutable en local y en la nube.
- Un **security gate** con Inspector que puede bloquear el deploy por vulnerabilidades.

Equivalencias en otras plataformas

Servicio AWS	Equivalente en el mercado
CodeBuild	GitHub Actions, Jenkins, CircleCI, GitLab CI
CodePipeline	GitHub Actions (workflow), Azure DevOps Pipelines
AWS Inspector	Snyk, Trivy, Aqua Security, Dependabot
CloudWatch Logs	Datadog, Splunk, ELK Stack

Mensaje para llevar

El concepto es el mismo en todas las plataformas: un archivo de configuración, un runner que lo ejecuta y un orquestador que decide cuando. Si aprendiste CodeBuild y CodePipeline, entiendes GitHub Actions en una hora. Los nombres cambian; la idea no.